

Desarrollo de un sistema de programación automatizada de tareas de mantenimiento en la industria minera

This is a subtitle

Emilio Bravo Maturana

¹ROA, Maastricht University

²SBE, Maastricht University

3 de enero de 2025



Maastricht University



Agenda

- ① Resumen Ejecutivo
- ② Análisis y descripción del problema
- ③ Definiciones teóricas
- ④ Aplicación de los conceptos teóricos al problema
- ⑤ Diseño de la solución
- ⑥ Resultados y análisis
- ⑦ Conclusiones y pasos a seguir

Section 1. Resumen Ejecutivo

Importancia del Mantenimiento Eficiente

- El mantenimiento eficiente de equipos es crucial para garantizar la continuidad operativa en cualquier industria.
- La programación manual de actividades de mantenimiento:
 - Es compleja e intensiva en tiempo.
 - Es propensa a errores debido a múltiples variables.
 - Depende altamente de personal especializado.

Propuesta de Automatización

Objetivo Principal

Automatizar la programación de tareas de mantenimiento mediante herramientas avanzadas de modelado y optimización.

- El problema se modela como un Problema de Programación de Proyectos con Restricciones de Recursos (*RCPSP*).
- Se utilizan técnicas de Programación por Restricciones (*Constraint Programming, CP*) para:
 - Minimizar el *makespan*.
 - Reducir tareas no programadas.

Impacto de la Solución

- **Optimización del tiempo y esfuerzo:** Liberación del equipo de programación de tareas manuales intensivas.
- **Reducción de errores:** Especialmente aquellos derivados de programaciones deficientes.
- **Resultados obtenidos:**
 - Cronograma generado para 4.898 tareas.
 - Programación del 98,6 % de las tareas.
 - *Makespan*: 21,5 días.
 - Tiempo de procesamiento: 6,12 segundos.

Section 2. Análisis y descripción del problema

Visión General

- **Objetivo:** Generalizar el proceso de programación de tareas en el rubro minero.
- **Metodología:**
 - Análisis de documentos normativos del proceso.
 - Encuesta dirigida al personal encargado de la programación.
- Este análisis permitió:
 - Comprender el funcionamiento operativo actual.
 - Identificar barreras y desafíos.
 - Validar la necesidad de una solución automatizada.

Descripción General del Proceso

- El proceso de programación de tareas se sustenta en la planificación previa, que define las ventanas temporales iniciales.
- El equipo de programación:
 - Ajusta las tareas a las restricciones reales de recursos.
 - Itera semanalmente para mantener el cronograma actualizado.
- **Elementos clave:** Ventanas temporales, disponibilidad de recursos, coordinación con áreas involucradas, criticidad de tareas.

Proceso de Planificación

- Responsables de definir ventanas temporales y requerimientos de mantenimiento.
- Consideraciones:
 - Objetivos estratégicos de la mina.
 - Tiempos de procesos industriales.
 - Otros factores del negocio minero.
- **Resultado:** Una planificación inicial en bruto (ventanas y recursos necesarios), pero sin contemplar restricciones reales de disponibilidad.

Proceso de Programación

- Recibe la planificación inicial como *input* para ajustar a las capacidades reales.
- Tareas principales:
 - **Generar el borrador inicial:** Fecha y hora de inicio para cada tarea.
 - **Resolver conflictos de recursos:** Evitar sobreasignaciones.
 - **Validar con otras áreas:** Asegurar viabilidad del cronograma.
 - **Realizar ajustes finales:** Incorporar cambios de último minuto.
- El cronograma resultante detalla tareas, fechas, duración y recursos, priorizando la continuidad operativa.

Ciclo Semanal de Programación

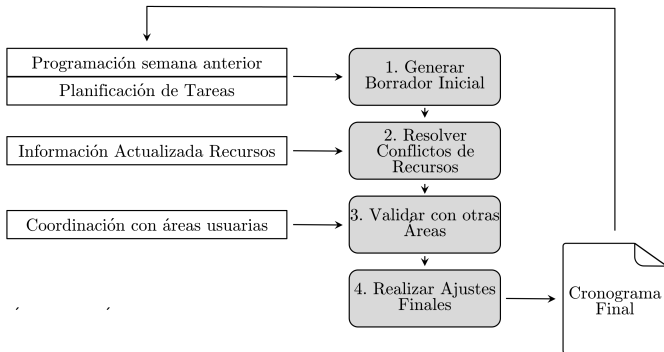


Figura: Ciclo Semanal de Programación de Actividades.

Diagnóstico del Proceso Actual

- **Encuesta** al personal de programación:
 - 64 % invierte más de la mitad de su jornada en la generación de cronogramas.
 - La fase más intensiva es la **creación del borrador inicial**.
 - El error más recurrente es la **no disponibilidad de materiales y repuestos**.
- **Entorno dinámico**: 90 % afronta cambios de último minuto al menos una vez por semana.
- Existen barreras de **coordinación** y **colaboración** con otras áreas.

Distribución del Tiempo de Programación

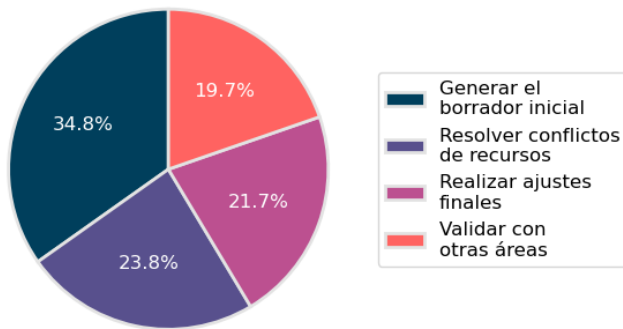


Figura: Distribución del tiempo de programación

Fuente: Elaboración propia en base a encuesta

Section 3. Definiciones teóricas

Definiciones Teóricas

- Se presentan los conceptos fundamentales para comprender el enfoque del proyecto.
- Temas abordados:
 - RCPSP (Problema de Programación de Proyectos con Restricciones de Recursos)
 - CP (Programación por Restricciones)
- Estos conceptos son esenciales para la metodología de programación de actividades de mantenimiento.

***RCPSP*: Definición e Importancia**

- El *RCPSP* es un problema clásico de investigación operativa y gestión de proyectos.
- Pertenece a los *Constraint Satisfaction Problems* (CSP) y, más específicamente, a los *Constraint Optimization Problems* (COP).

Características principales del *RCPSP*

- **Restricciones de precedencia:**

- Algunas actividades no pueden comenzar hasta que otras hayan finalizado.

- **Recursos limitados:**

- Mano de obra, equipos y materiales con disponibilidad limitada cada periodo.

- **Objetivo de optimización:**

- Generalmente, minimizar el tiempo total del proyecto (*makespan*).

- También pueden considerarse otros objetivos (costos, carga de recursos, etc.).

Ejemplo de *RCPSP*

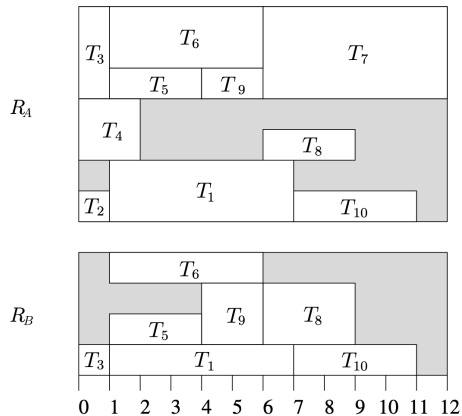


Figura: Ejemplo de un *RCPSP* con 2 recursos y 10 tareas.

Complejidad del *RCPSP*

- El *RCPSP* es un problema *NP-Hard*, lo que implica un crecimiento exponencial del espacio de soluciones conforme aumenta el número de tareas y recursos.
- Para instancias pequeñas, se pueden aplicar métodos exactos; sin embargo, a mayor escala, se requieren enfoques alternativos.
- **Programación por Restricciones (CP)** se presenta como una herramienta eficaz para modelar y resolver el *RCPSP* de manera declarativa.

***CP*: Definición e Importancia**

- *CP* es una técnica para resolver problemas combinatorios complejos (planificación, programación, optimización).
- En este trabajo, *CP* es fundamental para abordar el *RCPSP*, permitiendo modelar restricciones de tiempo y recursos.

CSP y COP

- Un CSP (*Constraint Satisfaction Problem*) se define por:
 - Un conjunto de variables V con dominios de valores D .
 - Un conjunto de restricciones R .
- La solución a un CSP es la asignación de valores a las variables que satisfaga todas las restricciones.

Esquema de Programación por Restricciones

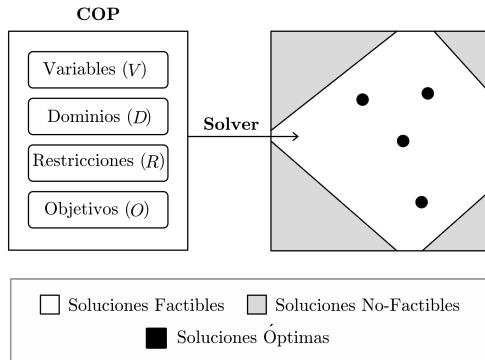


Figura: Esquema de Programación con Restricciones.

CP en el contexto del *RCPSP*

- Variables: Modelan inicio, duración y fin de cada actividad.
- Restricciones:
 - Temporales (precedencia).
 - De recursos (capacidades máximas).
- Ventanas temporales: Representan los plazos operativos de cada tarea.
- Función objetivo: Minimización del *makespan* (o adaptación a necesidades específicas).
- Ventaja: **Separación declarativa** entre el modelo y los algoritmos, facilitando la extensión y modificación del enfoque.

Section 4. Aplicación de los conceptos teóricos al problema

Aplicación de los conceptos teóricos

- En esta sección se describe cómo los conceptos de *RCPSP* y *CP* se aplican al diseño e implementación de la solución.
- Se detallan:
 - Datos de *input*.
 - Consideraciones al programar.
 - Generalización del problema como modelo matemático.
- Objetivo: Proveer un panorama completo de cómo se integran las bases teóricas con la realidad operativa y las restricciones de la empresa.

Datos necesarios para la programación

- El *input* principal proviene del proceso de **planificación**, que define:
 - Ventana de fechas.
 - Duración y criticidad de cada tarea.
 - Recursos requeridos (cuadrillas, equipos).
- También se incorporan datos desde el sistema *ERP*:
 - Turnos de las cuadrillas.
 - Disponibilidad de equipos auxiliares.
- Disponibilidad de **materiales y repuestos** no se considera debido a la falta de sistematización en el *ERP*.

Ejemplo de Tarea y sus atributos

Atributo	Descripción
ID Orden de Trabajo	4000001
ID Tarea	001
Descripción OT	Cambio de suspensión trasera lado izquierdo
Descripción Tarea	Retiro de suspensión trasera
Fecha Inicio Extrema	2024-11-29
Fecha Requerida	2024-12-30
Cuadrilla	BM001
Equipo Auxiliar	Alza Hombres Telescópico
Duración (horas)	6
Criticidad	2
Cantidad de Trabajadores	4

Consideraciones al programar

- Cada **tarea** incluye:
 - Fecha de inicio extrema y requerida.
 - Duración y criticidad.
 - Recursos requeridos (cuadrillas, equipos).
- **Restricciones de precedencia** dentro de la orden de trabajo (OT).
- **Cuadrillas:**
 - Tienen esquemas de turnos (disponibilidad limitada).
 - Cuentan con una capacidad (número de trabajadores) que determina cuántas tareas en paralelo pueden atender.
- **Equipos auxiliares** son recursos unitarios (no se comparten en simultáneo).
- **Cronograma base:** Se parte de cronogramas anteriores para ajustarlos e incorporar nuevas tareas.
- Si se programa una tarea de una **OT**, se programan todas las tareas de dicha OT.

Modelado como un *RCPSP*

- Objetivo: Asignar tiempos de inicio a cada tarea, minimizando el *makespan* y la cantidad de tareas no programadas.
- Restricciones de:
 - Capacidad de recursos.
 - Ventanas de tiempo y precedencia.
 - Intervalos prohibidos.
- **Resultado:** Un modelo matemático que integra todas las consideraciones de la realidad operativa.

Parámetros del modelo

- **Tareas** $T = \{T_0, T_1, \dots, T_n\}$:
 - Duración $C(T_i)$.
 - Recursos requeridos $D(T_i)$ y cantidad q_i .
 - Criticidad Impact_i .
 - Ventana de tiempo $W(T_i) = [\text{early}_i, \text{late}_i]$.
- **Recursos** $R = \{R_0, R_1, \dots, R_m\}$:
 - Capacidad $P(R_j)$.
 - Intervalos prohibidos $I(R_j)$.
- **Grupos de tareas** $G = \{G_0, G_1, \dots\}$:
 - Conjunto de tareas con restricciones de precedencia internas.

Variables de decisión

- $x_i \in \{0, 1\}$: Indica si la tarea T_i es **programada**.
- $S_i \in \mathbb{N}$: Momento de **inicio** de la tarea T_i .
- $E_i = S_i + C(T_i)$: Momento de **finalización** de la tarea T_i .
- $\text{Intervalo}(T_i) = [S_i, E_i)$: Intervalo de ejecución.
- $y_k \in \{0, 1\}$: Indica si el **grupo de tareas** G_k se programa.

Restricciones (I)

1 Ventanas de tiempo:

$$x_i = 1 \implies \text{early}_i \leq S_i \leq \text{late}_i - C(T_i)$$

2 Capacidad de recursos:

$$\sum_{\substack{T_i \in T \\ R_j \in D(T_i)}} q_i \cdot \delta_{ij}(t) \leq P(R_j), \quad \forall t \in \text{Horizonte}$$

donde $\delta_{ij}(t) = 1$ si $t \in [S_i, E_i]$ y $x_i = 1$, en caso contrario $\delta_{ij}(t) = 0$.

Restricciones (II)

3 Intervalos prohibidos:

$$x_i = 1 \implies \forall R_j \in D(T_i), \forall [a_k, b_k) \in I(R_j) : [S_i, E_i) \cap [a_k, b_k) = \emptyset$$

4 Restricciones de precedencia en grupos:

$$y_k = 1 \implies \forall (T_i, T_{i+1}) \in G_k : S_{i+1} \geq E_i \quad \text{y} \quad \forall T_i \in G_k : x_i = y_k$$

5 Consistencia de variables:

$$x_i = 0 \implies S_i = 0, \quad E_i = 0$$

Función objetivo

- Se desea minimizar *makespan* y penalizar la no programación de tareas.
- Definimos un **peso** para cada tarea basado en su criticidad:

$$w_i = (\text{Impact}_{\max} + 1 - \text{Impact}_i)^3$$

donde $\text{Impact}_{\max} = 3$.

- **Función objetivo:**

$$\text{mín } Z = \alpha \cdot \text{makespan} + \beta \cdot \sum_{i=0}^n w_i \cdot (1 - x_i)$$

→ α : Peso asociado al *makespan*.

→ β : Peso asociado a la no-programación de tareas (según criticidad).

Detalle de la función objetivo

① Componente Makespan ($\alpha \cdot \text{makespan}$):

- Minimiza la extensión global del horizonte temporal.
- Promueve la eficiencia en el uso de recursos.

② Componente Penalización ($\beta \cdot \sum w_i(1 - x_i)$):

- Permite que ciertas tareas no se programen, pero **penaliza** fuertemente la no-programación de tareas críticas.
- Evita que el modelo sea infactible si hay incompatibilidades extremas.

Balance de objetivos

- α y β controlan la preferencia entre un *makespan* menor y la programación de tareas críticas.
- $\alpha \gg \beta$: Favorece un *makespan* corto aunque implique no programar algunas tareas.
- $\beta \gg \alpha$: Preferencia por programar tantas tareas como sea posible (especialmente las críticas), aun si aumenta el *makespan*.
- Este balance permite al modelo adaptarse a distintas políticas operativas.

Section 5. Diseño de la solución

Diseño de la Solución

- El flujo de trabajo se organiza en 4 etapas secuenciales, desde la ingesta y preparación de datos hasta la generación de un cronograma optimizado.
- Cada etapa contribuye a estructurar y procesar eficientemente la información de programación.
- En la **Figura 5** se presenta un panorama general de este proceso.

Flujo de Trabajo de la Solución

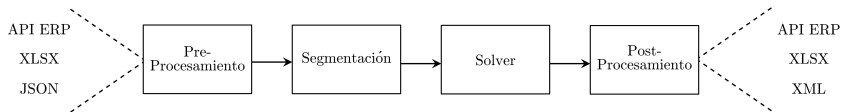


Figura: Flujo de Trabajo de la Solución.

- 1 Pre-Procesamiento
- 2 Segmentación en Subconjuntos
- 3 *Solver*
- 4 Post-Procesamiento

Pre-Procesamiento

- **Objetivo:** Unificar y limpiar la información procedente de diversas fuentes (*ERP*, *Microsoft Project*, u otros formatos).
- Operaciones clave:
 - **Corrección de discrepancias:** Ajuste automático de errores (ej. fechas incoherentes).
 - **Cuantización de ventanas horarias:** Expresar duraciones en intervalos de 15 minutos para optimizar el modelado.
 - **Restringir horarios fijos:** Configurar intervalos obligatorios para ciertas tareas.
 - **Estandarización en JSON:** Consolidación de datos (duraciones, recursos, dependencias) en un formato coherente.

Segmentación en Subconjuntos

- **Objetivo:** Reducir la complejidad del problema al dividirlo en grupos más pequeños.
- Método:
 - ① **Construcción del grafo:** Cuadrillas y equipos como nodos; existe arista si comparten recursos.
 - ② **Componentes conexos:** Identificación de grupos interrelacionados que se tratarán como un subconjunto.
 - ③ **Formación de subconjuntos:** Cada componente conexo se resuelve de manera independiente en el *solver*.

Ejemplo de Segmentación

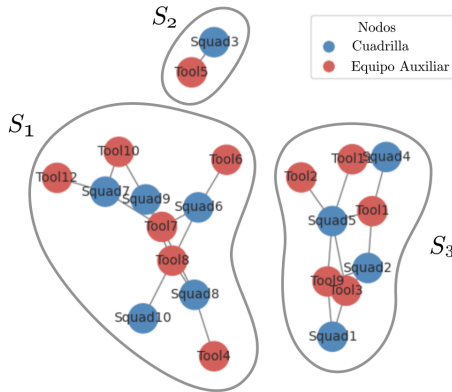


Figura: Ejemplo de segmentación a través de componentes conexos.

- Disminución de complejidad computacional.

Solver: Construcción de input

- Para cada subconjunto, se crean varios diccionarios en formato JSON:
 - **Diccionario de tareas:** Duración, recursos requeridos, etc.
 - **Diccionario de ventanas de tiempo:** Límites de inicio y fin.
 - **Diccionario de capacidades de recursos:** Máxima disponibilidad.
 - **Diccionario de agrupaciones:** Conjuntos de tareas (OTs).
 - **Diccionario de intervalos prohibidos:** Bloqueos por recurso.
 - **Diccionario de horarios preferenciales:** Mantener horarios previos en caso de cronograma existente.
- Uso de *OR-Tools* (CP-SAT solver) para modelar y resolver el *RCPSP*.
- **add_hint:** Permite incorporar soluciones anteriores o ajustes del usuario como punto de partida.

***Solver:* Ejecución y Paralelización**

- Cada subconjunto se resuelve de forma independiente, pudiendo ejecutarse en paralelo (*joblib*, hilos, múltiples núcleos).
- **Ventajas de la paralelización:**
 - Aprovecha mejor los recursos de hardware.
 - Reduce el tiempo total de ejecución.
- Se obtiene una solución parcial por cada subconjunto, que luego se unifica.

Post-Procesamiento

- **Objetivo:** Consolidar y preparar la salida del *solver* para su integración y análisis final.
- Salidas principales:
 - **Archivos XML para MS Project:** Permite incorporar el cronograma a entornos de gestión existentes.
 - **Archivo XLSX (Gantt):** Visualización rápida de la programación en hojas de cálculo.
 - **Integraciones con APIs:** Conexión automática con sistemas *ERP* o herramientas empresariales.

Conclusiones sobre el Diseño de la Solución

- **Flujo de trabajo modular:** Facilita la adaptación a diferentes fuentes de datos y objetivos.
- **Proceso escalable y paralelo:** Permite manejar grandes volúmenes de tareas sin sacrificar rendimiento.
- **Resultado final útil:** Archivos e integraciones que aportan valor a los usuarios finales en su día a día.

Section 6. Resultados y análisis

Resultados y análisis

- Se presentan los resultados clave del modelo de optimización.
- Se analiza:
 - El conjunto de datos utilizado.
 - El ajuste de parámetros en la función objetivo.
 - El impacto de la segmentación en subconjuntos.
- Beneficios operativos proyectados para el equipo de programación.

Conjunto de datos: Características generales

- Datos reales provistos por la empresa colaboradora.
- Período de planificación: Del 17 de julio al 3 de agosto de 2023.

Atributo	Valor
Número total de tareas	4.898
Número de órdenes de trabajo (OT)	1.055
Fecha mínima	2023/07/17
Fecha máxima	2023/08/03
Duración promedio (horas)	1,92
Número de cuadrillas	31
Número de equipos auxiliares	16

Cuadro: Resumen de las características del conjunto de datos.

Distribución de tareas según criticidad

- **Criticidad 1 (Alta):** 666 tareas (13,6 %)
- **Criticidad 2 (Media):** 2.403 tareas (49,1 %)
- **Criticidad 3 (Baja):** 1.829 tareas (37,3 %)

Recursos:

- 31 cuadrillas (distintos turnos y capacidades).
- 16 equipos auxiliares (recursos unitarios).

Distribución de Tareas por Nivel de Criticidad

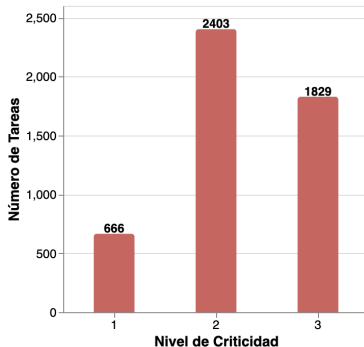


Figura: Distribución de tareas según criticidad.

Ajuste de α y β en la función objetivo

- La función objetivo combina:
 - α : Peso del *makespan*.
 - β : Penalización por no programar tareas (enfocada en criticidad).
- Múltiples experimentos con distintas combinaciones (α, β) para analizar:
 - Número total de tareas programadas.
 - Tareas no programadas por criticidad.
 - *Makespan*.
 - Tiempo de solución.

Resultados Generales (α - β)

Conf.	α	β	Tareas Prog.	Tareas no Prog.	Tiempo Sol. (s)	Makespan
(a)	1	0	0	4.898	3,43	0
(b)	0	1	4.854	44	4,35	1.072
(c)	1	1	4.658	240	8,42	492,75
(d)	1	2	4.831	67	9,95	516,25

Cuadro: Resultados con diferentes configuraciones de α y β .

Distribución de Tareas no Programadas por Criticidad

Conf.	Criticidad 1	Criticidad 2	Criticidad 3
(a)	666	2.403	1.829
(b)	0	8	36
(c)	21	59	160
(d)	0	12	55

Cuadro: Tareas no programadas, según criticidad, en cada configuración.

Conclusiones:

- (a): Minimiza *makespan*, pero no programa ninguna tarea.
- (b): Maximiza tareas programadas (incluye todas las críticas), pero *makespan* muy alto.
- (d): Equilibrio entre ambas metas: 4.831 tareas programadas y 516,25 h de *makespan*.

Segmentación en Subconjuntos

- Se dividió el total de 4.898 tareas en 14 subconjuntos.
- Reducción de complejidad y paralelización del *solver*.

Subc.	Tareas	% del total	Tiempo Sol. (s)
S1	2.598	53 %	4,34
S2	674	14 %	0,65
S3	319	7 %	0,43
S4	254	5 %	0,12
S5	198	4 %	0,01
S6	136	3 %	0,01
S7	122	2 %	0,00
S8	110	2 %	0,01
S9	98	2 %	0,00
S10	87	2 %	0,00
S11	84	2 %	0,01
S12	78	2 %	0,00
S13	72	1 %	0,00
S14	68	1 %	0,00

Tiempo total y conclusiones de la segmentación

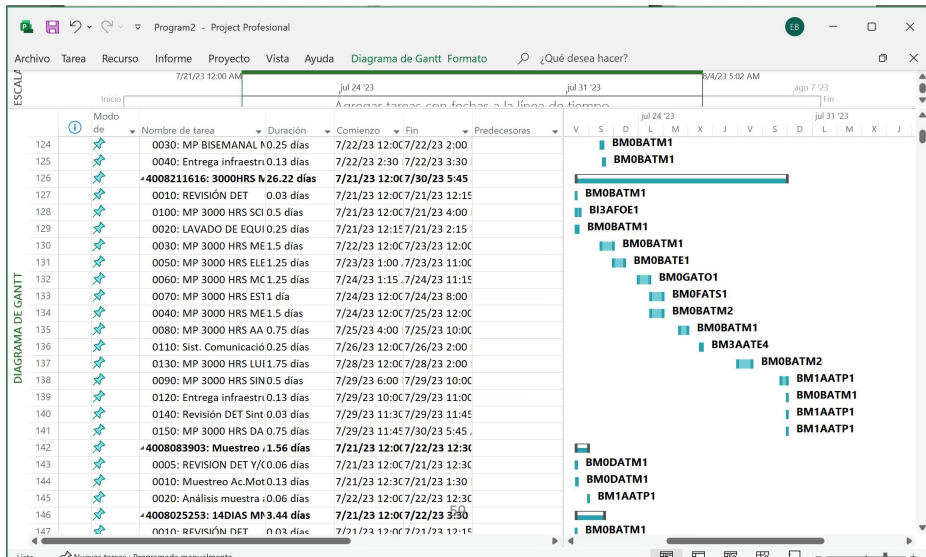
- El tiempo de solución total equivale al máximo de todos los subconjuntos:
 $\sim 4,34$ s.
- Paralelización:
 - Cada subconjunto se procesa en hilos o núcleos distintos.
 - Aprovechamiento óptimo del hardware disponible.
- Subconjuntos más pequeños ($< 5\%$) se resuelven casi instantáneamente.
- **Conclusión:** El enfoque modular y paralelo disminuye significativamente el tiempo total de ejecución sin sacrificar la calidad del cronograma.

Section 7. Conclusiones y pasos a seguir

Conclusiones Generales

- Se alcanzaron los objetivos propuestos, logrando una **solución eficiente** para la programación automatizada de mantenimiento.
- En un conjunto de 4.898 tareas:
 - 98,6 % de tareas programadas (solo 67 no programadas).
 - **Todas** las tareas de criticidad 1 fueron incluidas.
 - *Makespan*: 21,5 días.
- **Tiempo total de cálculo:** 6,12 segundos (incluye pre-procesamiento, segmentación, resolución y post-procesamiento).
- En la **Figura 8** se muestra un ejemplo de cronograma final en *Microsoft Project*.

Cronograma Resultante en MS Project



Impacto sobre proceso de programación

- **Automatización del borrador inicial:** Ahorro de $\sim 35\%$ del tiempo dedicado al cronograma ¹.
- **Ajustes finos al cronograma:**
 - Fijar tareas en horarios específicos.
 - Modificar disponibilidad de recursos.
 - Incorporar tareas de último minuto.
- Reducción del tiempo en resolución de conflictos de recursos ($\sim 24\%$) y ajustes ($\sim 22\%$) ².
- **Estimación global:** Se reduce en $\sim 57,6\%$ el tiempo total de creación y ajuste del cronograma.

¹Ver Anexo ??

²Ver Anexo ??

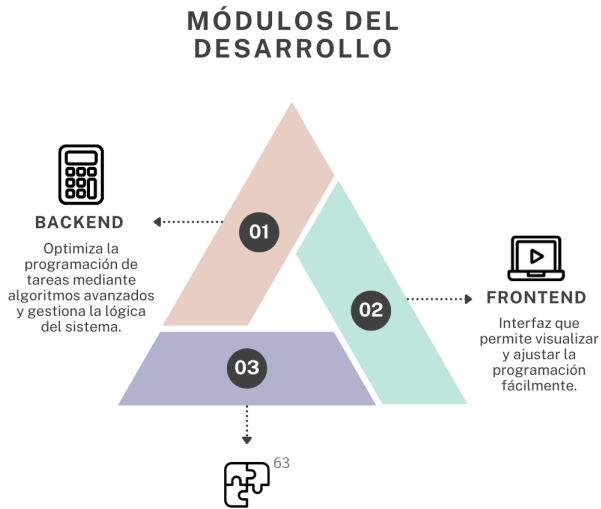
Limitaciones y consideraciones

- **Disponibilidad de materiales y repuestos:**
 - Principal error detectado.
 - No sistematizado en el *ERP*, por lo que no fue incluido en la solución.
- Para integrar este aspecto se requiere:
 - Paso previo de **gobernanza de datos**.
 - Procesos robustos de manejo en bodega.
 - Sistematización de información de repuestos.
- Sin estos pasos, la herramienta no reduce errores por falta de materiales, requiriendo colaboración y compromiso de otras áreas.

Pasos a seguir: Visión general

- **Objetivo:** Llevar la solución de un prototipo a un entorno operativo real.
- Módulos principales:
 - ① **Backend:** *OR-Tools*, *FastAPI*, *PostgreSQL*, contenedores *Docker*.
 - ② **Frontend:** *React.js*, librerías de visualización (*Gantt*, *D3.js*).
 - ③ **Integración:** Conexión bidireccional con ERP y sistemas existentes.

Módulos de Desarrollo de la Solución



Recomendaciones para el Backend

- **Framework:** *FastAPI* para exponer endpoints REST.
- **Motor de optimización:** *OR-Tools* (CP).
- **Base de datos:** *PostgreSQL* para almacenar OT, recursos y restricciones.
- **Despliegue:** Contenedores *Docker* + *Kubernetes* (opcional) para escalabilidad y alta disponibilidad.

Recomendaciones para el Frontend

- **Interfaz intuitiva:** Orientada a programadores de la empresa.
- **Tecnología:** *React.js* + librerías de visualización (p.ej. *React-Gantt-Chart*).
- **Funciones clave:**
 - Carga de órdenes de trabajo (integración ERP o subida manual).
 - Visualización y edición de cronogramas.
 - Exportaciones a *MS Project* y *Excel*.

Integración con Plataformas de la Empresa

- **ERP** y sistemas de gestión: Importación de órdenes de trabajo y disponibilidad de recursos.
- **Exportación:** Formatos compatibles (*XML* para MS Project, *XLSX* para Excel).
- Se recomienda:
 - Integración bidireccional.
 - Herramientas de administración de *APIs* para monitorear conexiones.
- **Beneficio:** Flujo de información continuo y actualizado, adaptado a la infraestructura de la empresa.

Reflexiones finales

- La solución mejora significativamente la automatización de la programación de mantenimiento.
- Aporta:
 - **Optimización de flujo de trabajo.**
 - **Reducción de errores.**
 - **Mayor eficiencia operativa.**
- La implementación de los pasos a seguir garantiza un **impacto tangible** en la empresa colaboradora.

Bibliography